

Security in Software Applications

Aliasing



SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

Today

- The joys of aliasing
- Alias control
- Confined Types
 - one of the proposed scheme for alias control
 - paper on Confined Types by Vitek & Bokowski to read
- Immutability
 - another way to avoid aliasing problems

Reminder of the problem

Security flaw in JDK1.1.1

```
package java.lang; public class
    Class {
        private Identity[] signers;
        /** Obtain list of signers of given class */
        public Identity[] getSigners() {
            return signers;
        }
    }
```

- Problem : unwanted aliasing

Aliasing

- **Safe** languages such as Java and C# do not have **pointer arithmetic** (eg `&x; x*`, ...) but they do have **references** and allow **aliasing**
- **aliasing** = having more than one reference to the same object

Desirable aliasing

- Efficiency
 - in **time**: e.g. alias to middle of list in list traversal
 - in **space**, by sharing object
- Consistent view to shared objects
- Useful programming patterns
 - e.g. callbacks, observer pattern, ...
- For "**objects**"/"**things**" in the informal sense of the word, as opposed to **values**, using references make sense
 - e.g. files, windows, USB sticks,
 - e.g. for anything that is hard in functional programming

Undesirable aliasing

- side-effects hard to control and understand
 - hard to debug
 - hard to maintain object invariants
 - hard to ensure consistent access
 - esp. when *concurrency* can cause race conditions
- aliases allow to by-pass interface operations
 - i.e. break encapsulation boundaries
 - aka representation exposure

This can cause problems if you have several pieces of code, some of which are more trusted than others, because untrusted code may exploit this

Representation exposure: spot the problems

```
class IntSet {  
    private int[] c; // the elements in the set, without  
        duplicates  
    IntSet() { c = new int[0]; }  
    IntSet(int[] a) { c = a; }  
    int getSize() { return c.length; }  
    boolean contains(int i) { ...}  
    void add(int i) { if (contains(i)) return;  
        newc = new int[c.length+1];  
        System.arraycopy(c,0,newc,0,c.length);  
        newc[c.length] = i;  
        c = newc;}  
    void remove(int i){...}  
    int[] getcontents() { return c;}
```

Representation exposure

N.B. two ways to unwanted alias of internal representation:

- **exporting** a reference

```
int[]    getcontents() { return c;}
```

- **importing** a reference

```
IntSet(int[] a) { c = a; }
```


Alias control

- Many systems have been – and are being – proposed for **alias control** (aka **ownership**, **encapsulation**, **confinement**,...)
 - whole workshops dedicated to this, eg IWACO'2009 at ECOOP'2009
- Many of these require annotations to express the programmer's intent

- Especially, things like: in

```
class A {  
  private B ref;  
  ....  
}
```

is **ref** allowed to be/meant to be aliased or not?

Alias control

```
public class BankTransfer{  
    private BankAccount from,to;  
    private int[] amount;  
    private char[] description;  
    ...  
}
```

Which fields do you think are (not) meant to be aliased?

Alias control

Several alias control type systems use notion of **rep** field

```
public class BankTransfer{ private
    BankAccount from,to;
    private @rep int[] amount;
    private @rep char[] description;
    ...
}
```

- rep field is **owned** by the object
- Important **design decision**, about the programmer's intention, which should be made **explicit**
 - for documentation
 - for type checking

(Note this decision also affects implementation of `.clone` and `.equals`)

Alias control

Several alias control type systems use notion of **rep** field

```
public class BankTransfer{ private
    BankAccount from,to;
    private @rep int[] amount;
    private @rep char[] description;
    ...
}
```

Idea: objects of type **@rep int[]** and of type **int[]** are not *assignment compatible*

Question: should **a.amount** and **b.amount** be assignment compatible for different Banktransfer objects **a** and **b**?

Question: could this solve the signers problem?

Alias control

- Many type systems have been – and are still being – proposed for alias control, e.g.
 - **universes** by Peter Muller
 - fine-grained alias control per object
 - **confined types** by Jan Vitek and Boris Bokowski
 - coarser alias control per package
- Design challenge: defining a system that is
 - **flexible and expressive** enough to capture typical programming patterns,
 - **simple** enough to be understandable and statically checkable

Immutability

- Another way to prevent aliasing problems:

- make objects **immutable**

Why does this prevent aliasing problems?

- About 660 (20%) classes in the Java standard library are immutable
 - Prime example: **java.lang.String** is immutable.

NB many (trusted) API methods take/produce strings as arguments/result. *(Why is immutability important then?)*

- The new programming language **Scala** clearly distinguishes immutable objects and treats them in special ways
 - encouraging a more functional programming style

Advantages of immutability

- Immutable objects can be aliased without problems
- They are immune to race conditions
- Conceptually simple (they are *values*)

- Because immutable objects can be safely shared, this can *improve efficiency*
 - because sharing objects saves the cost of copying them
- However, using immutable objects can also *decrease efficiency*
 - because not being able to change requires the creation of new objects for every change
 - eg think of using StringBuffer vs String

Advantages of immutability

- Could we use immutability to solve the problem with signers?

```
public class Class {  
    private Identity[] signers;  
    /** Obtain list of signers of given class */  
    public Identity[] getSigners() {  
        return signers;  
    }  
}
```

We could, if we had immutable arrays...

Advantages of immutability

Using immutability in the BankTransfer example: instead of

```
public class BankTransfer{  
    private BankAccount from,to;  
    private int[] amount;  
    private char[] description;
```

we would get

```
public class BankTransfer{  
    private BankAccount from,to;  
    private Integer amount;  
        // Integer objects are immutable, so no worries  
    private String description;  
        // String objects are immutable , so no worries
```

Better still, of course

Even better

```
public class BankTransfer{  
    final private BankAccount from,to;  
    final private Integer amount;  
        // Integer objects are immutable  
    final private String description;  
        // String objects are immutable  
    ...  
}
```

now BankTransfer objects are also immutable

- NB **final** is easy to forget but crucial for immutability in multi-threaded context; see slides on Language-Based Security!

Immutability

- Immutability of classes is an important property that deserved to be **made explicit** in code, and **checked**

Eg

```
public @Immutable class BankTransfer{ ...}
```

- Proposal for this

Immutable Objects for a Java-like Language

Christian Haack, Erik Poll, Jan Schäfer, and Aleksy Schubert,
ESOP'07& ECOOP'09

- uses notions of rep fields and anonymity, plus more

immutability and the need for rep

- We need to know which fields are rep-fields, ie. part of the object representation, in order to know what immutability means.
- E.g.

```
public @immutable class A{  
    final private @rep int[] a;    // a and contents of a immutable  
    ...  
}
```

vs

```
public @immutable class A{  
    final private int[] a;    // contents of a can mutate  
    ...  
}
```

immutability and the need for anonymous

```
public @immutable? class Integer {  
    public static Integer latest; // latest integer object allocated  
    private final int i;  
    public Integer(int j){ i = j;  
                           latest = this;}  
    public int getValue(){ return i;}  
}
```

Are these integer objects immutable?

- No, not in a multi-threaded context:
Constructor leaks **this**, hence field **i** not immutable:
Integer(5) may be observed to change from 0 to 5.

Recap

- Language constructs such as public/private/... enable protection of access to fields, methods, classes
- but do *not* provide a way to protect **integrity** of objects in system of interacting objects
- **Confined Types**
 - a particular scheme for alias control, that can provide integrity
 - all package scoped objects should be confined?
- **Immutability**
 - another way to avoid/reduce aliasing problems
 - built-in in newer programming languages such as Scala